

✓ Experiment

```
!pip install gdown
!gdown --folder 1-DD0bTJWo2PeXTURo0Y02RN7aqi0b5A7
```

[Show hidden output](#)

```
import pandas as pd
import os
import glob

# Find directory
data_dir = glob.glob('/content/*Water Quality Monitoring Dataset*')[0]

# Load each dataset into separate variables
df_england = pd.read_csv(os.path.join(data_dir, 'England_dataset.csv'))
df_combined = pd.read_csv(os.path.join(data_dir, 'Combined_dataset.csv'))
df_canada = pd.read_csv(os.path.join(data_dir, 'Canada_dataset.csv'))
df_china = pd.read_csv(os.path.join(data_dir, 'China_dataset.csv'))
df_usa = pd.read_csv(os.path.join(data_dir, 'USA_dataset.csv'))
df_ireland = pd.read_csv(os.path.join(data_dir, 'Ireland_dataset.csv'))

print("Loaded")
print(f"England : {df_england.shape}")
print(f"Combined : {df_combined.shape}")
print(f"Canada : {df_canada.shape}")
print(f"China : {df_china.shape}")
print(f"USA : {df_usa.shape}")
print(f"Ireland : {df_ireland.shape}")

# df_canada.info()
# df_canada.describe()
# df_canada.columns

df = df_combined

# # Count data for each 'Area' and order from most to least frequent
# area_counts = df['Area'].value_counts()
# print(area_counts)

# Mengurutkan berdasarkan frekuensi dari tinggi ke rendah
area_counts = df['Area'].value_counts()

# Menampilkan hanya 20 area teratas
top_20_areas = area_counts.head(20)
print(top_20_areas)
```

Advanced Soft Sensor BOD: Optimized Hou Bay Analysis

In this section, we implement a professional-grade pipeline for BOD prediction using Hyperparameter Tuning (Grid Search) and advanced preprocessing techniques.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Data Preparation & Outlier Handling
# Re-filtering for Hou Bay to ensure clean slate
df_hou_adv = df_combined[df_combined['Area'] == 'Hou Bay'].copy()
features = ['Ammonia (mg/l)', 'Dissolved Oxygen (mg/l)', 'Orthophosphate (mg/l)',
target = 'Biochemical Oxygen Demand (mg/l)'

data_adv = df_hou_adv[features + [target]].dropna()

# Removing extreme outliers using IQR to improve model stability
Q1 = data_adv[target].quantile(0.25)
Q3 = data_adv[target].quantile(0.75)
IQR = Q3 - Q1
# Using a liberal multiplier to keep most data but remove extreme noise
data_clean = data_adv[~((data_adv[target] < (Q1 - 3 * IQR)) | (data_adv[target] > (Q3 + 3 * IQR)))]

print(f"Original samples: {len(data_adv)}")
print(f"Samples after outlier removal: {len(data_clean)}")
```

```
Original samples: 45997
Samples after outlier removal: 44986
```

```
# 2. Splitting and Scaling
X_adv = data_clean[features]
y_adv = data_clean[target]

X_train_adv, X_test_adv, y_train_adv, y_test_adv = train_test_split(X_adv, y_adv,

scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train_adv)
X_test_scaled = scaler.transform(X_test_adv)
```

```
# 3. Hyperparameter Tuning: Random Forest
print("Tuning Random Forest...")
rf_param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5]
}

grid_rf = GridSearchCV(RandomForestRegressor(random_state=42), rf_param_grid, cv=5)
grid_rf.fit(X_train_scaled, y_train_adv)

best_rf = grid_rf.best_estimator_
print(f"Best RF Params: {grid_rf.best_params_}")
```

```
Tuning Random Forest...
Best RF Params: {'max_depth': None, 'min_samples_split': 2, 'n_estimators': 200}
```

```
# 4. Hyperparameter Tuning: XGBoost
print("Tuning XGBoost...")
xgb_param_grid = {
    'n_estimators': [100, 200],
    'learning_rate': [0.05, 0.1],
    'max_depth': [4, 6],
    'subsample': [0.8, 1.0]
}

grid_xgb = GridSearchCV(XGBRegressor(random_state=42), xgb_param_grid, cv=3, scoring='r2')
grid_xgb.fit(X_train_scaled, y_train_adv)

best_xgb = grid_xgb.best_estimator_
print(f"Best XGB Params: {grid_xgb.best_params_}")
```

```
Tuning XGBoost...
Best XGB Params: {'learning_rate': 0.1, 'max_depth': 6, 'n_estimators': 200, 'subsample': 0.8}
```

```
# 5. Comprehensive Evaluation
def model_report(model, X, y_true, name):
    preds = model.predict(X)
    r2 = r2_score(y_true, preds)
    rmse = np.sqrt(mean_squared_error(y_true, preds))
    mae = mean_absolute_error(y_true, preds)
    print(f"--- {name} Report ---")
    print(f"R2 Score: {r2:.4f}")
    print(f"RMSE      : {rmse:.4f}")
    print(f"MAE       : {mae:.4f}\n")
    return preds
```

```
y_pred_rf_opt = model_report(best_rf, X_test_scaled, y_test_adv, "Optimized Random Forest")
y_pred_xgb_opt = model_report(best_xgb, X_test_scaled, y_test_adv, "Optimized XGBoost")
```

```
--- Optimized Random Forest Report ---
```

```
R2 Score: 0.4058
```

```
RMSE      : 0.4020
```

```
MAE       : 0.2879
```

```
--- Optimized XGBoost Report ---
```

```
R2 Score: 0.3634
```

```
RMSE      : 0.4161
```

```
MAE       : 0.3029
```

```
# 6. Visualizing Residuals and Prediction Quality
```

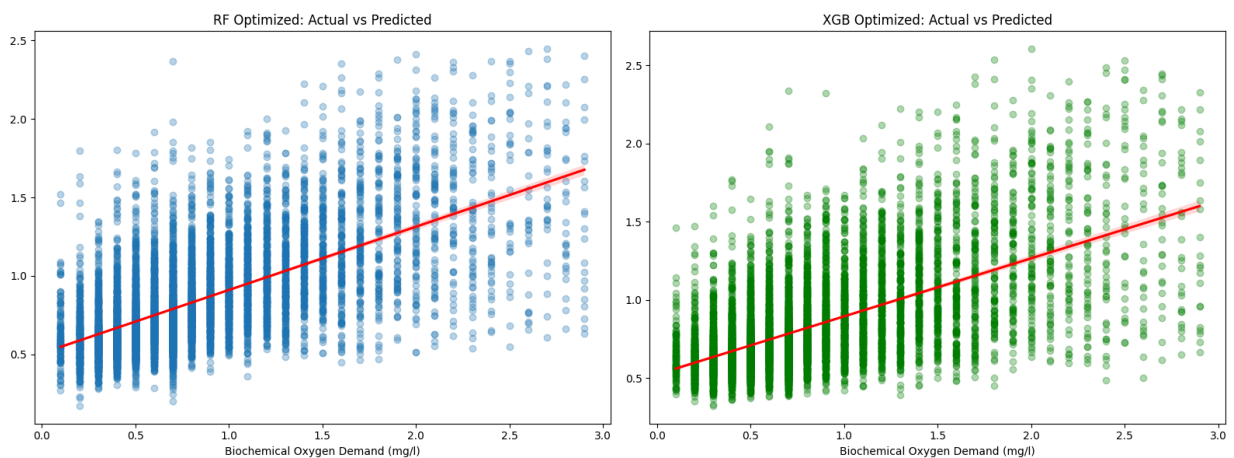
```
fig, ax = plt.subplots(1, 2, figsize=(16, 6))
```

```
sns.regplot(x=y_test_adv, y=y_pred_rf_opt, ax=ax[0], scatter_kws={'alpha':0.3}, line_color='red')
ax[0].set_title("RF Optimized: Actual vs Predicted")
```

```
sns.regplot(x=y_test_adv, y=y_pred_xgb_opt, ax=ax[1], scatter_kws={'alpha':0.3}, line_color='green')
ax[1].set_title("XGB Optimized: Actual vs Predicted")
```

```
plt.tight_layout()
```

```
plt.show()
```



> Soft Sensor BOD – Area: Hou Bay

Pada bagian ini kita akan membangun model prediksi BOD menggunakan fitur kimia air lainnya khusus untuk data dari Hou Bay.

🔍 ↳ 4 cells hidden

>